AM5600: Computational Methods in Mechanics

Term Project Report

Advait Vinay Dhopeshwarkar(CE17B024)

Imperialist Competitive Algorithm

Table of Content:

Sr. No.	Title	Pg. No.
1	Introduction	3
2	The Natural Process and The Algorithm	3
3	Algorithm Flowchart	6
4	Algorithm Description	7
4.1	Initialization	7
4.2	Assimilation	8
4.3	Revolution	8
4.4	Calculation of Power of each Empire	9
4.5	Imperialistic Competition	9
4.6	Elimination	10
4.7	Convergence and optima	10
5	Python Implementation	11
6	Benchmark tests (2 tests)	11
7	Scope for improvement	16
8	References	17

Introduction

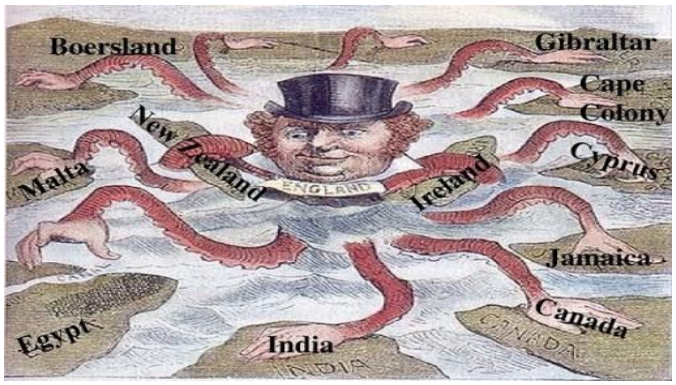
In this term paper, we discuss and learn about the Imperialist Competitive algorithm(ICA). ICA is a type of population based, gradient free algorithm like the Genetic Algorithm(GA) and is used for solving different types of optimization problems. A general goal of the ICA is obtaining the global minima of a given function. We know that any optimization problem can be converted into a minimization problem and thus we can use ICA to solve most optimization problems.

In this paper, we discuss the idea and the motivation for the algorithm, in terms of the natural/social phenomenon. We then discuss the general workings of the algorithm and then implement the algorithm in Python. We then test our code against a standard benchmark problem and compare our results against those which were found in the research paper. We will also discuss the results obtained and draw our conclusions.

The Natural Process and the Algorithm

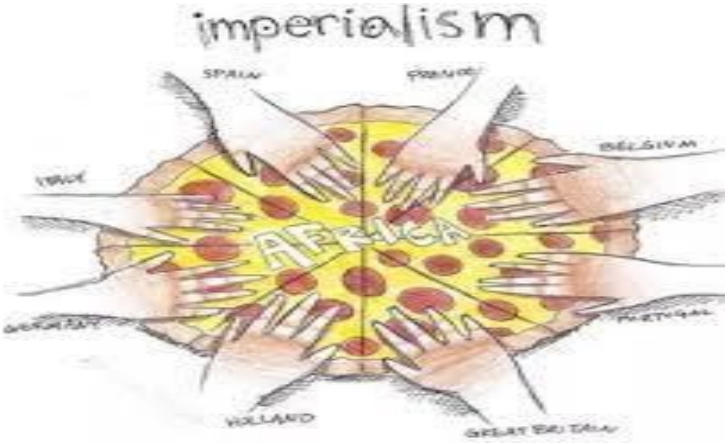
The ICA was inspired by the human social evolution, more specifically Imperialistic Competition and evolution of empires. Here we discuss this natural process and the key ideas that we use in the algorithm based them.

Imperialism refers to the policy of extending the power and rule of the government beyond its own boundaries. We have seen in the history of human social evolution that empires have always tried to extend their control beyond its boundaries and gain power. The European empires like the British, French, Dutch, Portuguese and the Spanish are the prime examples of this.



An imperialist may dominate other countries(colonies) by direct rule or by indirect means such as controlling the market for goods and raw materials. The example of the direct rule can be the control of the British empire over the Indian subcontinent. The example of the indirect rule

of the British and the Portuguese empires over the Chinese by controlling the market and trade in Hong Kong and Macau respectively, but there never direct control over the entire region. The main goal of the imperialists was to extend their control over larger and larger territory, gain more wealth and most importantly more power. The imperialists of different empires also competed with each other for power and the fear of losing out on the economic gains. For example the French, British, Dutch and the Portuguese competed for control of the Indian subcontinent. They even fought wars against each other for control. For example the French supported Tipu Sultan to counter the British in India and also supported the American revolution which eventually helped the USA to gain freedom from British colonial rule.



The competition among the empires resulted in formation of some large power empires and even the collapse of the weaker ones. For example, years of war and imperialistic competition led to economic downfall of the French, which created divide among the masses(classes) in France and eventually resulted in the storming of the 'bastille' and execution of Louis XVI.

We know that imperialists try to influence their colonies both culturally and socially and try to assimilate the colonies with their culture.

For example the British changed the education system in the sub-continent by introducing western ideas and subjugating the Indian culture and education system. This served the British well as they got the local workforce that followed their orders.

Assimilation however also resulted in anger among the colonies and created a will to fight for freedom. Thus, colonies often challenged the power of the imperialist and even revolted against the imperialist to gain freedom.

The ideas obtained from historical notes as discussed above, are used to formulate the ICA algorithm. We will now look at the algorithm and draw parallels to our discussion:

Problem setup:

1. In the ICA, our world(domain of the function) consists of a bunch of points which we will henceforth refer to as countries.
2. We then define a notion of power based on the value of the function at each of the countries. Since we are minimizing, lower value of function implies higher power.
3. These countries are divided into imperialists and colonies depending on their power. That is, countries with the highest powers(about 10 out of 100 countries) become imperialists.
4. Each Imperialist controls an empire and the remaining countries are distributed among these empires depending on the power of the imperialist which controls it. That is, a more power imperialist controls a larger empires(larger number of colonies).

Iteration:

In each of the iterations we repeat the following process until convergence is achieved:

1. As discussed earlier, imperialists have a major influence on their colonies. Thus, colonies of each empire assimilate toward their corresponding imperialist.
2. While assimilating however, the colonies may gain power and revolt against their imperialist to gain control of the corresponding empire. This step is termed as revolution. If a particular colony turns out to be more powerful than their corresponding imperialist, it replaces the imperialist and becomes the new imperialist of the empire.
3. Further, each Empire competes with all remaining empires for more territory(ie. more colonies). In the algorithm, all the empires compete for the control of the weakest colony of the weakest empire in our world.
4. Thus, the weakest colony of the weakest empire is reassigned to a more powerful empire.

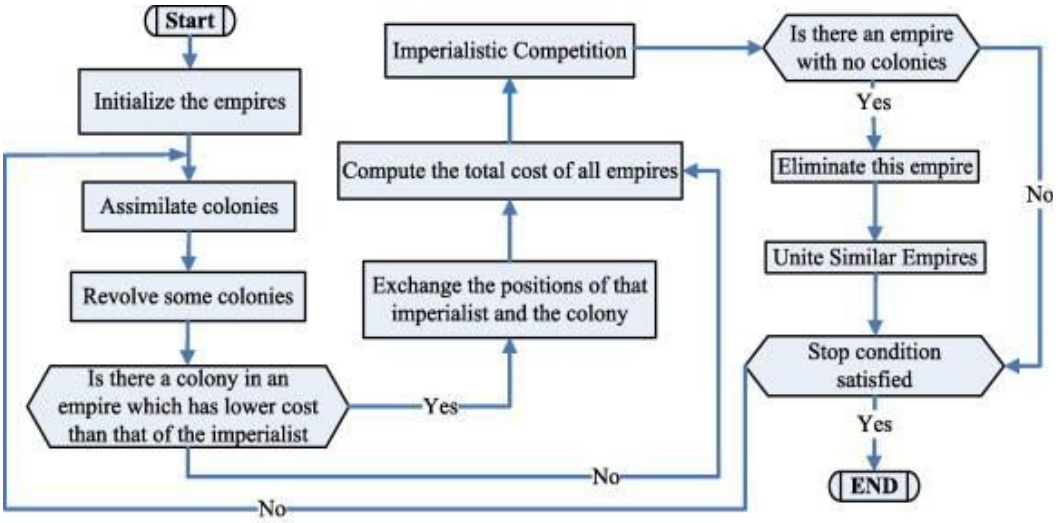
5. In doing so, the weakest empire might run out of colonies eventually. In this case we claim that the empire is powerless and we eliminate such an empire. (The survival of the fittest empire)

Convergence and optima:

1. We repeat the iteration procedure over and over until all but one empires are eliminated. Thus, this is our convergence criteria.
2. The last remaining empire is the fittest empire and the imperialist of this last remaining empire is the fittest country.
3. Thus, the point corresponding to the imperialist of the last remaining empire gives us the optima.(Rule of the fittest)

Algorithm Flowchart

The following chart briefly describes the steps of the ICA as discussed previously:



Algorithm Description

Now we will discuss each of the step in detail:

Initialization:

1. Let us assume that we are trying to optimize a function of two variables f(x,y).
2. In such a case we would define a country as a point in the X-Y plane. We randomly create about 100-200 such countries.
3. Associated cost, Ci=f(country)
4. Normalized Cost, Ci'=Ci-max{Cj's}.
5. Power, Pi=|Ci'/(∑Cj')|
6. We then select the top 8-10 countries from the list of countries with the highest power and call them imperialists.
7. Each imperialist is assigned an empire and the remaining countries are distributed among these empires as colonies.
8. The distribution is done based on the power of the corresponding imperialist. That is, a more powerful imperialist is assigned with a proportionally larger number of colonies.

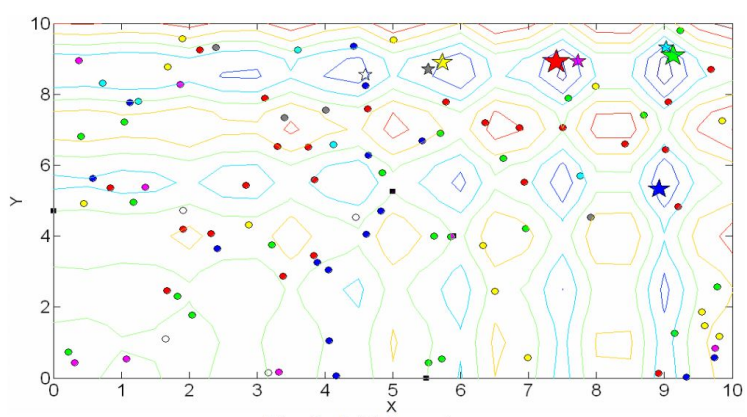
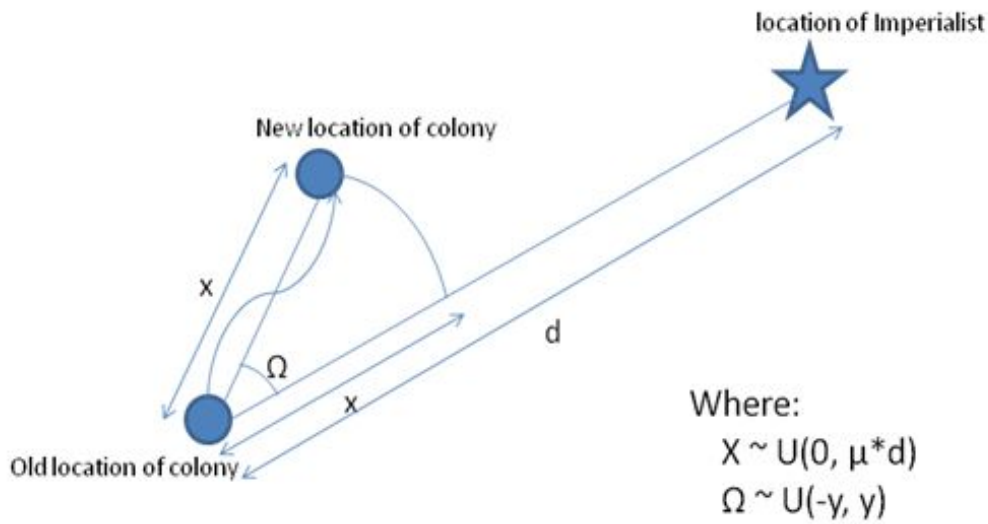


Fig. 9. Initial empires.

Assimilation:

1. In this step we move the colonies in each of the empires closer to their corresponding imperialist as shown in the figure below. Note that U(a,b) indicates a uniformly distributed random variable over [a,b] interval.



2. It was observed in the benchmark examples that the algorithm converged well for **y= π /4** and **μ=2**.
3. The randomness in which the colony approaches the imperialist is what adds variation, similar to the "mutation" step that was discussed in GA.

Revolution:

1. In the Assimilation step, it is likely that a colony might land at a position where it becomes more powerful as compared to its corresponding imperialist.
2. In this case we make such a colony, the new imperialist of the empire and the old imperialist becomes a colony of that empire.

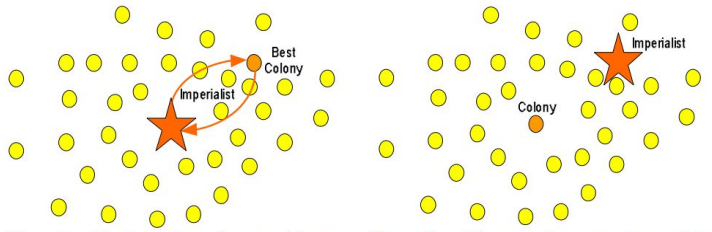


Fig. 5a. Exchanging the positions of a colony and the imperialist

Fig. 5b. The entire empire after position exchange

Calculation of Power of each Empire:

1. The power of the empire is mainly dependent on the power of its imperialist.

2. However, the colonies also make small contributions.

The following formula is used for calculation of power:

Power(Empire)=Power(Imperialist)+k*ΣPower(colonies)

where k<1(typically 0.001~0.1) .

Imperialistic Competition:

1. As discussed earlier, Empires try to expand their control over territory, that is, gain more colonies by competing.
2. For this, normalized power vector is calculated as follows:
   i. **P** =[p1, p2, p3, …., pn]
      1.
3. where pi refers to the normalized power of each remaining empire and is given by:
   i. pi = Pi/ΣPj , where Pi is as calculated in the previous slide.
4. Next, the weakest colony of the weakest empire is reassigned to more powerful empire
5. For reassignment, R is calculated as:
   i. R=[r1, r2, r3, …, rn], where rj~ U(0,1){uniform r.v.}
   ii. D = P – R
   iii. D = [p1-r1, p2-r2, p3-r3, …, pn-rn]
6. The empire with the greatest value of pi-ri is assigned with the weakest colony of the weakest empire.

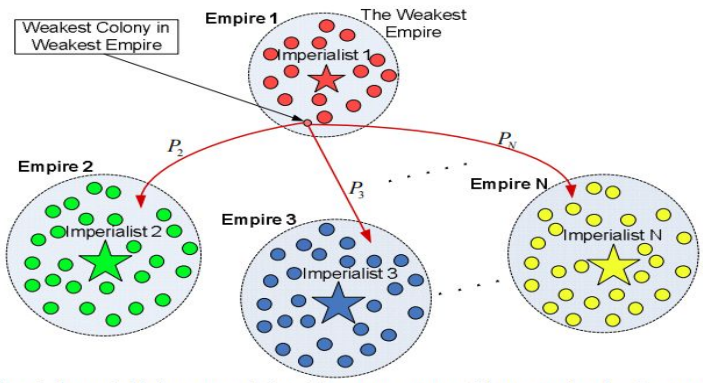


Fig. 6. Imperialistic competition. The more powerful an empire is, the more likely it will possess the weakest colony of the weakest empire.

Elimination:

1. Weak empires might lose all of their colonies through imperialistic competition to powerful empires.
2. In this case, the empire becomes powerless and is eliminated.
3. In doing so, only the fittest among the empires remain.

Convergence and optima:

1. Convergence Criteria: Repeat the above steps until only one empire remains.
2. Optima (result): Imperialist of the last remaining empire.(“The rule of the fittest”)

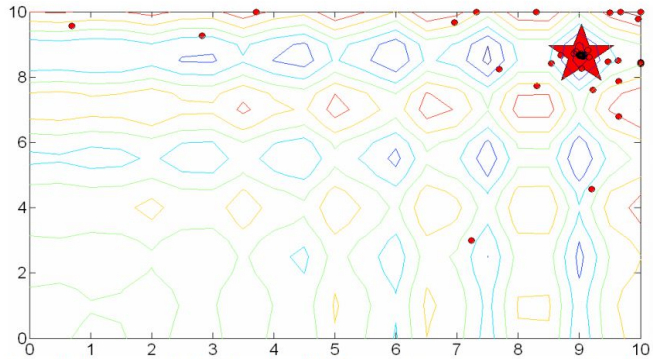


Fig. 12. Empires at iteration 15: All the countries have converged to the global minimum of the function being controlled by a unique empire.

Python Implementation

The algorithm has been implemented in python in the 'ProjectCode' folder. Run only the **'main.py'**, which contains the main implementation of the algorithm.

Suitable comments have been provided wherever required with brief explanation within the code itself.

The 'ProjectCode' folder contains four important files. These are as follows:

1. **'main.py' :** This contains the main implementation of the algorithm. Run code in this file to execute the algorithm. This uses the remaining three files so ensure that these are contained in the same directory.
2. **'Country.py' :** This defines the **Country** class. Check the code for the methods and attributes of this class.
3. **'empire.py' :** This defines the **Empire** class. Check the code for the methods and attributes of this class.
4. **'function.py' :** This contains the function that we are trying to optimize. The function may be changed in this file as per requirement.

Benchmark Tests and Remarks:

1. **T**his is a custom test case. Check out the second case for the standard case.

function.py > ...

```python
import math

# change the function here.
# fn is the function that we are optimizing.


def fn(items) -> int:
    '''
    items is a list of integers which define the coordinates
    returns an integer value.
    '''
    x = items[0]
    y = items[1]

    return -((x-4)**3)*(-1+math.e**(-x))+y**2
```

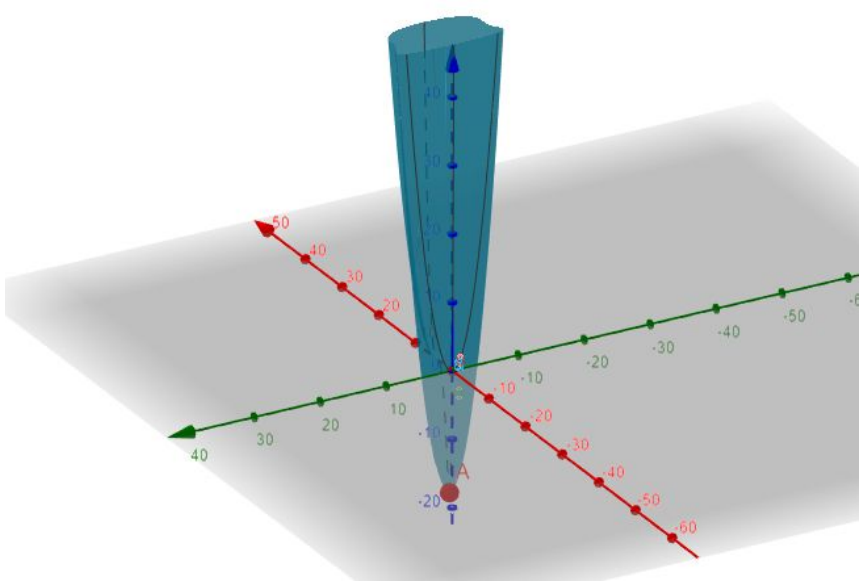
The function shown in the above figure was used as the benchmark function to test the implementation.

$f(x,y) = -(x-4)^3 . (e^{-x} - 1) + y^2$

The result obtained from the test run are shown in the screenshot of the terminal output below:

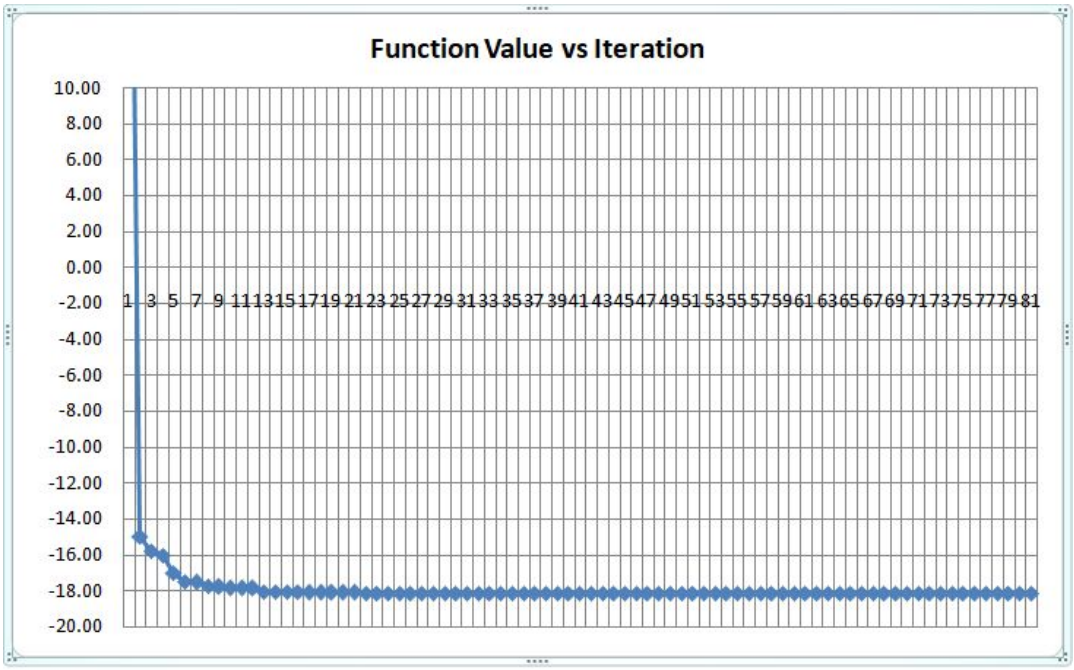
```
C:\Users\Advait_PC\Desktop\7TH SEM\CMM\TermProject\ProjectCode>main.py
optima:  [0.7361782190315147, 1.971598020986469e-09]
function value optima:  -18.116168566107305

C:\Users\Advait_PC\Desktop\7TH SEM\CMM\TermProject\ProjectCode>
```

The above result obtained when plotted along with the function, we can notice that it does indeed give the minima(indicated by the red point). The red point was manually plotted **(0.7361,0,-18.116)** to confirm the minima. Thus, we can validate our implementation.

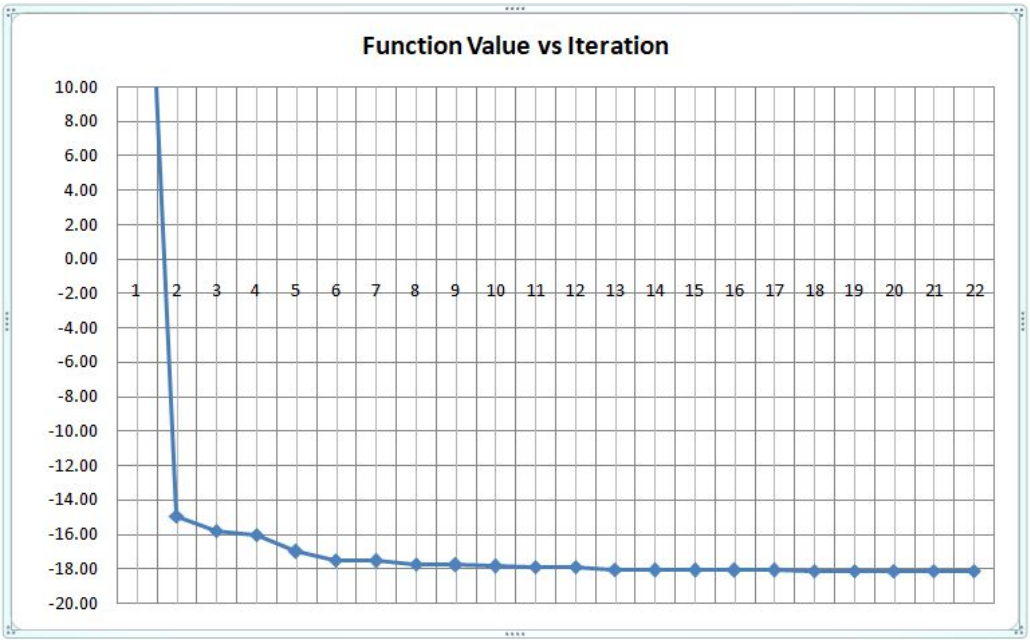


The following plot shows the cost of the most powerful empire as a function of the iteration. This plot may differ for different runs of the programs but the trend of the plots are very similar.

We can see that the convergence is achieved at about 81st iteration. We can note however that the imperialist of the most powerful empire achieves the optimal value much sooner.



We can notice that the imperialist of the most powerful empire achieves optima value within 22 iterations in the magnified plot below:



Thus, based on the observations above we can conclude that the implementation of the algorithm works correctly for the function considered.

2. **T**he following test case is a **standard** benchmark case. The 1st case was a custom test.

The following figure shows the function used for the second benchmark test case:

Problem G_3:

$$f=(x^2+y^2)^{0.25}\times\sin\left\{30\left[(x+0.5)^2+y^2\right]^{0.1}\right\}+|x|+|y|$$

$$-\infty<x,y>+\infty,\quad minimum:\ f(0,0)=0$$

function.py > ...

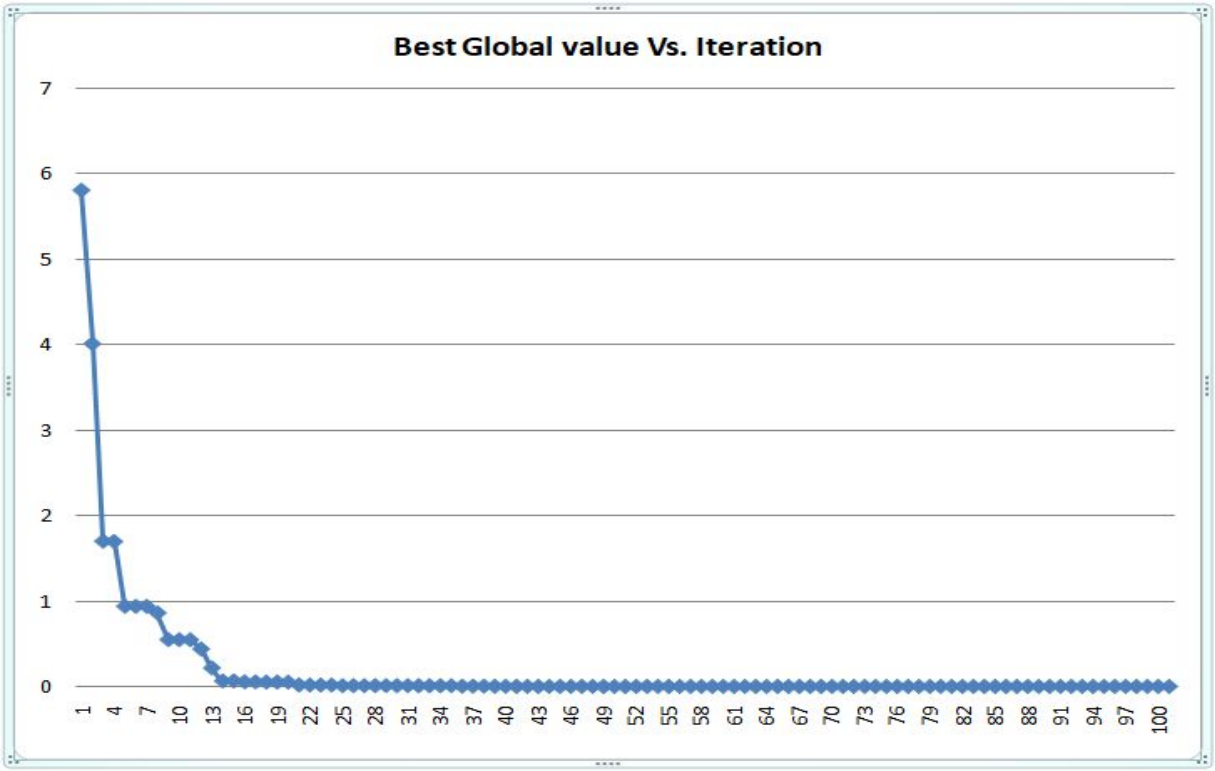
```python
import math

# change the function here.
# fn is the function that we are optimizing.


def fn(items) -> int:
    '''
    items is a list of integers which define the coordinates
    returns an integer value.
    '''
    x = items[0]
    y = items[1]

    return ((x*x+y*y)**(0.25))*math.sin(30*((y*y+(x+0.5)**2)**0.1)*(math.pi/180))+abs(x)+abs(y)

# -((x-4)**3)*(-1+math.e**(-x))+y**2

```

The following were the results obtained from our implementation:

```
C:\Users\Advait_PC\Desktop\7TH SEM\CMM\TermProject\ProjectCode>main.py
optima:  [4.621510514374117e-10, -9.856657473238696e-11]
function value optima:  9.569631599234584e-06

C:\Users\Advait_PC\Desktop\7TH SEM\CMM\TermProject\ProjectCode>
```

Clearly the results are as expected thus once again confirming the validity of the implementation.

The following figure shows the Best global value obtained as a function of number of iterations. Again we can observe that the optima is obtained within about 22-25 iterations. Without our optimization (discussed in scope for improvements), The algorithm does not terminate after 100 iterations but goes on until only one empire is left. This takes time without adding much value to our result. Thus we can observe that our small modification improves efficiency without giving up much on the accuracy.



Scope for Improvement

We observed in the benchmark example that the most powerful imperialist achieves optima much earlier than the algorithm termination through the convergence condition discussed earlier. This is a very important observation as we can see that we need not wait for the algorithm to eliminate all the empires, but we can get a reasonable answer within fewer iterations. This is a possible way in which we can make the algorithm faster.

I used the above idea to make small modifications to the code:

1. Instead of waiting for convergence to occur, a condition was imposed that the number of iterations should not exceed a given number(say 100).
2. So at the end of the given number of iterations, the algorithm terminates without convergence, but we can now find out the most powerful empire and the imperialist of this empire will give us an estimate which is very close to the true value.

*Note: The code submitted('ProjectCode') has this modification in it. And the 2nd Benchmark case uses this updated code for 100 iterations

There might be scope for further improvement in the efficiency by adjusting the limit on number of iterations, number of countries in the algorithm, initial number of empires, the value of y and Ω in the assimilation step etc. Note however that these values are not generalized but different values work best for different problems(functions). Thus, this could be thought of as a limitation of the ICA.

References

1. https://en.wikipedia.org/wiki/Imperialist_competitive_algorithm
2. Imperialist Competitive Algorithm: An algorithm for optimization inspired by imperialistic competition. Esmaeil Atashpaz-Gargari, Caro Lucas. 2007 IEEE Congress on Evolutionary Computation (CEC 2007)